

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	CSA0309	Course Title:	Data Structures
CO Assessed:	CO4 – Articulation (BL3)	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA	Total Marks:	100
CO4 Statement:	Apply hashing strategies for efficient data storage and sorting algorithms for arrangement of data.		
Student Name:	K. Kavya	Reg. No.:	1A2521262
Section / Batch:	1	Date:	28/08/2026

Code of Conduct

I K. Kavya (1A2521262) (Name / Reg No)

certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: [Signature]

Instructions

- This test contains 5 Comparative Analysis questions. Total: 100 Marks.
- When a student answer using comparative analysis should be based on four requirements: time complexity, space complexity advantages & limitations, real-world scenarios and operations.
- No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Hashing	Linear Probing & Separate Chaining	1	20
Sorting	Merge Sort & Quick Sort	2	20
Hashing	Quadratic Probing and Double Hashing	3	20
Sorting Algorithms	Complexity Analysis	4	20
Quick Sort	Recursive and Iterative implementations	5	20
GRAND TOTAL:			100 Marks

Annexure A - Comparative Analysis

1. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: (20 Marks)
 - a) Clustering effect
 - b) Memory usage
 - c) Performance under high load factor
 - d) Which method is more suitable for large-scale applications and why?
2. Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? (20 Marks)
3. Compare Quadratic Probing and Double Hashing in terms of: (20 Marks)
 - a) Clustering issues
 - b) Collision resolution efficiency
 - c) Suitability for dense hash tables
4. Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: (20 Marks)
 - a) Practical significance
 - b) Impact on algorithm selection
5. Compare Recursive and Iterative implementations of Quick Sort in terms of: (20 Marks)
 - a) Space complexity (stack usage)
 - b) Ease of implementation
 - c) Risk of stack overflow

Rubrics for Calculation

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Understanding	20	Demonstrates complete and accurate understanding of all data structures with advanced insights	Shows clear understanding with minor gaps	Basic understanding; some misconceptions present	Limited or incorrect understanding
Comparative Analysis Depth	20	Thorough, multi-dimensional comparison with strong justification and critical evaluation	Good comparison with relevant factors considered	Limited comparison; lacks depth or justification	Minimal or incorrect comparison
Use of Parameters (Time/Space/Operations)	30	All parameters analysed accurately with complexity discussion	Most parameters covered correctly	Few parameters considered; some inaccuracies	Parameters missing or incorrect
Critical Thinking	20	Strong justification of why one structure is better in given contexts	Some justification present	Limited reasoning	No critical reasoning
Clarity	10	Exceptionally well-structured, logical flow, clear tables/diagrams	Well-organized with minor issues	Some organization issues; lacks clarity	Poor structure and readability

Faculty In-charge	Course Coordinator	Program Director
Signature: 	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1. compare linear probing & separate chaining.

Introduction:

Hashing is a technique used to store and retrieve data quickly using a hash table. A hash function converts a key into a index in table. Sometimes, two or more keys produce the same index.

a) clustering effect:

Linear probing uses open addressing. When the calculated position is already occupied, it searches the next available position sequentially.

eg: index: 0 1 2 3 4

data: A B C - -

Separate chaining handles collisions by maintaining a linked list or a chain at each hash table index. Multiple elements with the same hash values are stored in the same chain.

b) memory usage:

Linear probing: uses less memory because elements are stored directly in hash table.

Separate chaining: requires extra memory for linked list nodes and pointers.

c) performance under high load factor

Linear probing: performance decreases significantly when the table becomes nearly full.

Separate chaining: handles high load factors better bc multiple elements can be stored in the same chain.

d) suitability for large scale application:

Separate chaining is generally more suitable for large scale application bc it handles collisions and high load factors efficiently

conclusion:

Linear probing saves memory, while separate chaining provides better flexibility and performance under heavy load.

20 Quick sort vs merge sort for 10 million LLM
Memory requirement:

Quick sort: can be performed in place, requiring less extra memory, but recursion uses stack space

Merge sort: usually requires extra space but for linked list, merging can be done efficiently w/ changing pointers.

cache behaviour.

Quick sort: work well with arrays because array elements are stored continuously giving good cache locality.

Merge sort: is better suited for linked list because it mainly performs sequential access

Time complexity:

Quick sort: best / Average - $O(n \log n)$; worst - $O(n^2)$

Merge sort: best / Average / worst - $O(n \log n)$.

Conclusion:

Merge sort is better for linked lists, while

Quick sort is often better for arrays.

3. Quadratic probing vs double hashing

both are open addressing methods used to handle collisions in hash tables.

a) clustering issues:

Quadratic probing: reduces primary clustering but can still have secondary clustering.

Double hashing: provides better distribution and greatly reduces clustering.

b) collision resolution efficiency.

Quadratic probing searches positions using a quadratic sequence such as $1^2, 2^2, 3^2$.

double hashing: uses a second hash function to determine the step size, difference of key.

c) suitability for dense hash tables

when the hash table becomes dense, collisions increase. Quadratic probing may become less efficient while double hashing usually maintains better distribution.

conclusion:

Double hashing is generally preferred for dense hash tables because it provides better collision distribution.

4. Best case vs worst case time complexity

a) practical significance:

Best case analysis shows how efficient an algorithm can be used ideal conditions. worst case analysis helps determine

eg.:

Best case : $O(n \log n)$

worst case : $O(n^2)$

Merge sort has:

Best case : $O(n \log n)$

worst case : $O(n \log n)$

b) Impact on Algorithm selection:

Best case complexity is useful when input is usually favorable. worst case complexity is important when predictable performance is required.

eg.:

insertion sort $\rightarrow$ Best : $O(n)$

worst : $O(n^2)$

merge sort $\rightarrow$ Best }
worst } : $O(n \log n)$

Conclusion:

Best case shows ideal performance, while worst case helps determine the maximum possible running time. Both should be considered when selecting an algorithm.

Recursive vs Iterative Quick sort

Quick sort can be implemented using recursion or an explicit stack.

a) Space complexity

Recursive: uses the function call stack. Average space is $O(\log n)$, but worst case can reach $O(n)$.

Iterative: uses a manually managed stack and avoids recursive function calls.

b) ease of implementation:

Recursive: Easier to write and understand because it directly follows the divide and conquer approach.

Iterative: Avoids recursive stack overflow because recursion is not used.

c) Risk of stack overflow:

Recursive: has a higher risk of stack overflow because it is highly unbalanced.

Iterative: Avoids recursive stack overflow because recursion is not used.

Conclusion:

Recursive quick sort is simpler and easier to implement, while iterative quick sort provides better control over stack usage & is safe for large inputs.